

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**Национальный исследовательский ядерный университет «МИФИ»**

---



**Институт интеллектуальных кибернетических систем  
КАФЕДРА КИБЕРНЕТИКИ (№ 22)**

## **План управления конфигурацией**

**Б20-524**

Корначук М.В.

Лукичев А.Н.

Кузнецов М.М.

Латыпов И.В.

**Москва 2023**

# Содержание

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| <b>1</b> | <b>Определения и термины</b>                         | <b>3</b>  |
| <b>2</b> | <b>Среда управления конфигурацией ПО</b>             | <b>5</b>  |
| 2.1      | Организация и ответственность . . . . .              | 5         |
| 2.1.1    | Роли, дающие права доступа к задаче: . . . . .       | 6         |
| 2.2      | Единицы конфигурации . . . . .                       | 6         |
| 2.3      | Трассировочная база данных . . . . .                 | 7         |
| <b>3</b> | <b>Описание мероприятий управления конфигурацией</b> | <b>9</b>  |
| 3.1      | Идентификация конфигурации . . . . .                 | 9         |
| 3.2      | Базовые версии и трассируемость . . . . .            | 9         |
| 3.3      | Регистрация проблем . . . . .                        | 12        |
| 3.4      | Управление изменениями . . . . .                     | 12        |
| 3.5      | Рассмотрение изменений . . . . .                     | 13        |
| 3.6      | Хранение, воспроизведение и выпуск . . . . .         | 14        |
| 3.7      | Контроль загрузки ПО . . . . .                       | 15        |
| 3.8      | Контроль среды ЖЦ . . . . .                          | 15        |
| 3.9      | Средства управления данными ЖЦ ПО . . . . .          | 15        |
| 3.10     | Учёт состояния конфигурации . . . . .                | 16        |
| <b>4</b> | <b>Хранение, воспроизведение и выпуск</b>            | <b>17</b> |
| <b>5</b> | <b>Критерии перехода</b>                             | <b>18</b> |

# 1. Определения и термины

1. **Commit:** Фиксация изменений в проекте, содержащая детали этих изменений, дату создания, уникальный идентификатор и идентификатор автора.
2. **Структура Commit'ов:** Древовидная организация commit'ов, где каждый commit может иметь одного или нескольких предшественников. Основной веткой в данной структуре является 'master'. Ветки могут объединяться, интегрируя изменения из обеих веток.
3. **Git:** Распределенная система управления версиями.
4. **GitHub:** Веб-платформа для совместной разработки, использующая Git для управления версиями.
5. **Конфликт слияния:** Ситуация, возникающая при попытке объединения двух веток, в которых были внесены различные изменения в одни и те же части кода.
6. **Pull Request:** Механизм для предложения изменений в основную ветку проекта на GitHub. Позволяет другим участникам проекта просмотреть и обсудить предлагаемые изменения перед их интеграцией.
7. **Отслеживаемые файлы:** Файлы, которые Git отслеживает для изменений. Текущие версии этих файлов добавляются в список изменений для следующего commit при использовании команды 'add'.
8. **Непрерывная интеграция (CI):** Практика разработки, при которой изменения в коде регулярно интегрируются в основную ветку проекта и проходят автоматическое тестирование с целью быстрого выявления и устранения ошибок.
9. **Blob** (в контексте Git): Blob в Git — это бинарное представление файла, идентифицируемое уникальным хешем. Оно содержит снимок содержимого файла без метаданных.
10. **Индекс** (в контексте Git): Это временная область, где Git хранит файлы, которые вы добавили с командой `git add`, которые затем станут частью следующего commit'a.
11. **Репозиторий** (в контексте Git): Хранилище для файлов проекта, их истории изменений и структуры commit'ов. Может быть локальным или удаленным. станут частью следующего commit'a.
12. **Issue:** Инструмент на GitHub для отслеживания задач, ошибок, предложений и проблем, с возможностью обсуждения и управления процессом разработки.
13. **Структура (Golang):** Пользовательский тип данных, который позволяет объединять данные различных типов. Структуры используются для моделирования объектов с набором

атрибутов. Каждый атрибут структуры называется полем, и у каждого поля есть свой тип.

14. **Функция (Golang):** Блок кода, который можно вызывать по имени. Она может принимать аргументы и возвращать значение(я). Функции используются для выполнения конкретных задач и могут повторно использоваться в разных частях программы.
15. **Интерфейс (Golang):** Набор сигнатур методов. Тип считается реализующим интерфейс, если он определяет все методы интерфейса. Интерфейсы обеспечивают гибкость и масштабируемость в коде, позволяя использовать различные типы, которые выполняют одни и те же операции.

## 2. Среда управления конфигурацией ПО

В рамках данного проекта для управления конфигурацией программного обеспечения используется GitHub. В GitHub хранится полная история коммитов, что позволяет использовать любой из них в качестве базовой версии.

Также GitHub хранит задачи и проблемы (Issue) и фиксирует их изменения, авторов и исполнителей. Задаче могут быть присвоены метки, отражающие её важность, а также прочие метки, определяющие к какой части проекта относится данная задача. Также задаче может быть присвоен статус, а сама задача может быть просмотрена в канбан доске, позволяющей визуализировать текущие задачи и их статус.

Кроме того GitHub хранит кандидатов в новые базовые версии (Pull request), их состояние. Также в PR хранится информация о том, кто является проверяющим данные изменения и комментарии к изменениям, оставленные в результате проверки.

При изменениях Issue или PR, всем участникам процесса разработки, которые с ними связаны (имеют роль participant), приходят уведомления на GitHub и на электронную почту.

### 2.1 Организация и ответственность

**Роли, дающие права доступа к репозиторию:**

**Владелец (Owner)**

- **Доступ:** Полный доступ к репозиториям, включая чтение и запись.
- **Администрирование:** Полные права управления и настройки проекта.

**Сопровождающий (Maintainer)**

- **Доступ:** Чтение и запись в репозиториях.
- **Администрирование:** Ограниченные права управления репозиториями.

**Разработчик (Developer)**

- **Доступ:** Просмотр и модификация кода в репозиториях.

**Гость (Guest)**

- **Доступ:** Отсутствует право просмотра репозитория.

## **Репортер (Reporter)**

- **Доступ:** Только просмотр репозитория.

### **2.1.1 Роли, дающие права доступа к задаче:**

#### **Автор задачи (Author)**

- **Обязанности:** Инициация задач и их описание.

#### **Создатель коммита (Committer)**

- **Доступ:** Редактирование и добавление изменений в репозитории.
- **Ограничения:** Возможны ограничения на запись в основную ветку.

#### **Исполнитель задачи (Assignee)**

- **Обязанности:** Реализация задач и создание запросов на слияние.

#### **Утверждающий (Approver)**

- **Обязанности:** Проверка и утверждение изменений перед слиянием в основную ветку.

#### **Участник процесса (Participant)**

- **Права:** Участие в дискуссиях и реализации задач. Может объединять в себе несколько ролей в рамках одной задачи.

## **2.2 Единицы конфигурации**

Единицы конфигурации (ЕК) являются отдельными элементами проекта, подлежащими трассировке и управлению на протяжении жизненного цикла разработки. Каждая ЕК имеет определенную последовательность версий, с уникальными идентификаторами. Вот основные ЕК, определенные для этого проекта:

- **Задача:** Представляется в виде issue на GitHub. Идентифицируется по уникальному номеру, автоматически присваиваемому при создании. Название задачи может не быть уникальным, однако такие ситуации следует избегать. Вместо классических версий, для задач существует история обсуждения.

- **Pull Request (PR):** Это запрос на слияние ветвей в GitHub. PR имеет уникальный номер и название. Всегда связан с задачей, решаемой в рамках проекта, и таким образом обеспечивает связь между задачами и запросами на слияние.
- **Требование:** Определено одной или несколькими строками в документе. Идентификация требований основана на стандартах разработки. Идентификаторы версий конкретного требования соответствуют версиям файла, в котором были внесены изменения.
- **Структура:** Описывается структурой языка Go с прикрепленными методами. Идентифицируется по названию. Версии структуры определяются на основе версий файла, в котором вносились изменения.
- **Функция:** Описывается кодом функции на языке Go. Функция идентифицируется через имя пакета и имя функции. Версии функции определены на основе версий файла, содержащего функцию, и в которых вносились изменения.
- **Артефакт проектирования:** Чаще всего представлен в виде диаграммы или схемы, сохраненной в отдельном файле. Идентификация артефакта основана на имени файла. Версия артефакта определяется идентификатором версии соответствующего файла.

## 2.3 Трассировочная база данных

Представляет собой базу данных SQLite, расположенную по пути `/trace/traceDB.sqlite3`. Содержит информацию о связях различных единиц конфигурации между собой.

Каждая запись в таблице этой базы хранит:

- Идентификатор родительской единицы конфигурации
- Тип родительской единицы конфигурации
- Версию родительской единицы конфигурации
- Отношение между родительской и дочерней единицами конфигурации
- Идентификатор дочерней единицы конфигурации
- Тип дочерней единицы конфигурации
- Версию дочерней единицы конфигурации

Список обозначений типов:

- SYSREQ - требования к системе, расположение: `/Specification/GeneralDescription`
  - SOFTREQ - требования к ПО, расположение: `/Specification/SpecificRequirements`
- и

Для удобства работы с базой данных рекомендуется создать исполняемый файл на языке Golang, который будет реализовывать все функции, а также предоставлять удобный интерфейс для взаимодействия с пользователем.

## Требования

- Golang 1.18+
- go-sqlite3
- tabulate

## Как использовать

- `trace ls [option] <relative_path>` - выводит список единиц конфигурации, записанных в файле, расположенному по относительному пути.
  - `-t` - выводит только те единицы конфигурации, которые находятся в трассировочной базе.
  - `-n` - выводит только те единицы конфигурации, которые не находятся в трассировочной базе.
- `trace add <from_unit> <from_type> <from_version> [relation <to_unit> <to_type> <to_version>]` - добавляет в трассировочную базу новую запись.
- `trace update <unit>` - обновляет версию указанной единицы конфигурации во всех записях трассировочной базы.
- `trace rel [options] <unit>` - выводит все записи из трассировочной базы, где участвует указанная единица конфигурации.
  - `-p` - выводит записи, в которых единица конфигурации является родительской.
  - `-c` - выводит записи, в которых единица конфигурации является дочерней.
- `trace der <relative_path>` - выводит производные единицы конфигурации.
- `trace smell [option|<relative_path>]` - проверяет актуальность данных в трассировочной базе.

## 3. Описание мероприятий управления конфигурацией

### 3.1 Идентификация конфигурации

Целью данного этапа является однозначное определение каждой единицы конфигурации (ЕК) и ее последующих версий. Это необходимо для установления структурированного управления ЕК и возможности ссылающегося на них.

В проекте используются следующие объекты для идентификации ЕК:

- **Файл.** Это объект, который может быть индексируем системой контроля версий git, и может быть как текстовым, так и бинарным. Для каждого файла уникальный идентификатор определяется его местоположением и именем. Версия файла определяется по хэшу последнего commit'а, в котором данный файл был изменен.
- **Репозиторий.** Это централизованное место для хранения данных, в данном проекте — это репозиторий на платформе GitHub, содержащий файлы и папки. Уникальный идентификатор репозитория — это его URL. Версия репозитория определяется набором идентификаторов последних commit'ов в каждой ветке.

При создании новых задач и Pull Requests идентификация производится автоматически средствами GitHub. Для всех остальных ЕК используются идентификаторы соответствующих файлов, идентификация которых осуществляется при помощи git при их добавлении или изменении.

Процесс идентификации файла и его версии следующий:

1. Участник команды разработчиков создает или модифицирует файл в локальной директории репозитория.
2. Файл добавляется в commit с использованием команды `git add`.
3. В директории `.git/objects/` создается блоб-файл с сжатым добавленным файлом.
4. Данный файл включается в индекс репозитория.

### 3.2 Базовые версии и трассируемость

#### Правила введения новых версий

1. **Базовая версия репозитория:** Определяется последним коммитом в главной ветке (master). Каждое новое изменение, добавленное в главную ветку, обновляет базовую версию, которая идентифицируется хэшем данного коммита.

2. **Версии единиц конфигурации:** Совпадают с базовой версией репозитория на момент внесения соответствующих изменений.
3. **Релизная версия проекта:** Вводится по завершении каждой итерации разработки. Определенному коммиту присваивается тег, обозначающий релизную версию. Нумерация версий соответствует стандарту Semantic Versioning 2.0.0, где номер версии имеет формат MAJOR.MINOR.PATCH.
4. **Версия для ревью:** Определяется последним коммитом в истории merge request, после которого убран статус "Work In Progress".
5. **Версия для слияния в master:** Определяется коммитом, соответствующим условиям версии для ревью, но после выполнения операции rebase.

Все версии идентифицируются уникальными хэшами коммитов, обеспечивая однозначное определение состояния проекта на каждом этапе его жизненного цикла.

## Контроль библиотек ПО

Для обеспечения надежности и стабильности системы отслеживания статусов кабинетов НИЯУ МИФИ, контроль за использованными библиотеками ПО осуществляется с помощью пакетных менеджеров.

1. **Golang:** В Golang используется система управления зависимостями *go mod*. В конфигурационном файле *go.mod* указываются все библиотеки, необходимые для проекта, и их версии. При компиляции проекта, система автоматически устанавливает необходимые версии библиотек. Библиотеки для Golang располагаются в индивидуальных git-репозиториях и загружаются командой *go mod download*. Пример содержания файла *go.mod*:

```
module module_name
go 1.17
require (
    github.com/sirupsen/logrus v1.8.0
    github.com/go-chi/chi v5.0.2
)
```

2. **Локальное хранение библиотек:** Исходный код всех используемых сторонних библиотек сохраняется локально. В репозитории на GitHub сохраняются только версии используемых библиотек, что обеспечивает возможность воспроизведения и согласованности проекта без необходимости хранения полного кода библиотек.

Все изменения и обновления версий библиотек проходят через систему контроля версий GitHub, что обеспечивает прозрачность и отслеживаемость всех изменений.

## Трассируемость

1. Базовые версии отдельных единиц конфигурации не устанавливаются. Вместо этого используется базовая версия репозитория в целом. С учетом того, что проект состоит из трех репозиториях: документация, серверная часть и клиентская часть, все трассировочные ссылки будут актуальными в рамках определенной версии всего репозитория. Применение нумерации версий по Semantic Versioning 2.0.0 обеспечивает синхронизацию версий всех репозиториях.
2. В проекте сохраняется информация о трассировке между:
  - Требованиями к системе и требованиями к ПО.
  - Требованиями к ПО и артефактами проектирования.
  - Артефактами проектирования и исходным кодом.
  - В некоторых случаях между требованиями к ПО и исходным кодом.
  - Требованиями к ПО и тестовыми процедурами.
  - Исходным кодом и тестовыми процедурами.
  - Issue и pull request на GitHub.
3. Трассировка между требованиями к системе, требованиями к ПО и артефактами проектирования осуществляется с помощью инструмента трассировки *trace.go*, расположенного в репозитории на GitHub. Документация к этому инструменту находится в том же репозитории.
4. Трассировка требований к ПО на исходный код не осуществляется.
5. Для обеспечения трассируемости исходного кода к артефактам проектирования и конкретным требованиям, в комментариях к коду указывается ссылка на соответствующий источник. Формат этой ссылки определен в стандарте кодирования.
6. Для обеспечения трассируемости требований к тестовым процедурам, в файле "traceDB.sqlite3" содержится информация о соответствии каждого требования определенной тестовой процедуре. Также указывается версия требования с точностью до конкретного commit'а.
7. Трассировка между исходным кодом и автоматически выполняемыми тестовыми процедурами основана на записи тестовых процедур.
8. Трассировка между исходным кодом и ручными тестовыми процедурами не требуется.
9. Трассировка между Issue и pull request устанавливается автоматически самим GitHub. В описании issue указываются ссылки на все связанные pull request, и наоборот.

### 3.3 Регистрация проблем

При обнаружении проблемы или предложении улучшений создается issue (задача/проблема) на GitHub с меткой `type::Suggestion`. Issue представляет собой запись о проблеме или предложении в рамках системы управления версиями GitHub (система управления версиями на GitHub).

Для создания issue, следует перейти в раздел `Issues` (Проблемы) репозитория проекта и нажать кнопку `New issue` (Создать задачу). Далее необходимо заполнить предоставленные поля в соответствии с инструкциями, описанными в данном документе, и подтвердить создание issue.

Issue оформляется по шаблону `Suggestion` (Предложение), который доступен при создании:

- **Краткое описание.** Краткое изложение обнаруженной проблемы или предложения.
- **Действия по воспроизведению.** Последовательность действий для воспроизведения проблемы.
- **Текущее ошибочное поведение.** Описание фактического результата.
- **Ожидаемое поведение.** Описание ожидаемого результата.
- **Логи и/или скриншоты.** Приложение соответствующих логов или скриншотов для иллюстрации проблемы.
- **Возможные исправления.** Если есть предположения, указание на участок кода, который может быть причиной проблемы.

Любые изменения в данных жизненного цикла ПО (Software Lifecycle), включая добавление новых функций, также регистрируются через issue. В этом случае описание проблемы оформляется по шаблону `Task` (Задание):

- **Расположение.** (Location) Ссылка на соответствующий раздел или файл.
- **Задача.** (Task) Полное описание задачи.
- **Дополнительно.** (Additional Info) Любая дополнительная информация или уточнения.
- **Зависимости.** (Dependencies) Указание на другие issue, которые необходимо решить перед началом работы над текущей задачей.

### 3.4 Управление изменениями

Этапы:

1. Ответственный за задачу (Assignee) ищет в списке текущих issue на GitHub те, в которых он указан в качестве Assignee. Issue создаются на этапе регистрации проблем.
2. Assignee присваивает выбранному issue метку `stage::InProgress`, указывая на

начало работы над задачей, и создает pull request (PR) с префиксом WIP (Work in Progress). Автоматически создается ветка для внесения изменений. Название этой ветки указано на странице PR.

3. Assignee вносит необходимые изменения в соответствии с описанием issue. Изменения могут быть внесены локально с последующим push в репозиторий.

При локальном внесении изменений:

- 3.1. Обновление локального репозитория командой `git pull`.
  - 3.2. Переключение на соответствующую ветку с помощью `git checkout <branch_name>`.
  - 3.3. Внесение и проверка изменений.
  - 3.4. Добавление измененных файлов в индекс с помощью `git add`.
  - 3.5. Создание коммита с сообщением, соответствующим шаблону Issue #<number>: <description>.
  - 3.6. Передача коммита в репозиторий с помощью `git push`.
4. После завершения всех изменений, Assignee убирает префикс WIP из названия PR, сигнализируя о готовности изменений к рассмотрению.

В случае возникновения проблем или неясностей в процессе работы, Assignee может изменить метку issue на `stage::Problem` и обратиться за помощью к другим участникам команды в комментариях к issue. Если необходимость внесения изменений становится сомнительной, метка важности может быть изменена на `priority::Optional`, и в комментариях к issue обсуждается возможность её закрытия или пересмотра.

### 3.5 Рассмотрение изменений

Процесс рассмотрения изменений определяется следующим образом:

1. После добавления коммитов в репозиторий, инструмент непрерывной интеграции (CI) инициирует сборку проекта и запускает автоматические тесты. Для репозитория документации выполняются следующие действия:
  - Преобразование markdown-файлов в latex-формат и компиляция соответствующих документов.
  - Анализ трассировки для проверки актуальности версий в трассировочной базе данных.
2. Ответственный за коммит (Committer) удостоверяется в успешности сборки. В случае ошибок в процессе сборки или тестирования, Committer должен либо внести корректировки, либо обратиться за помощью к коллегам.

3. После завершения всех изменений, Committer убирает префикс WIP из названия pull request, указывая на готовность к ревью.
4. Создатель (Author) регулярно проверяет состояние pull request'ов на GitHub. При появлении pull request, готового к ревью, Author назначает рецензента (Approver) для данного pull request. Committer не может быть назначен в качестве Approver.
5. Approver указывает на выявленные ошибки в комментариях к pull request. Если ошибки обнаружены, Approver добавляет префикс WIP к описанию pull request или использует команду /wip в комментарии к pull request.
6. Committer вносит необходимые изменения и добавляет их в pull request. Если Approver удовлетворен результатом, он нажимает кнопку Approve на pull request, указывая на готовность к интеграции изменений в основную ветку.
7. Уведомление о завершении ревью приходит на электронную почту Author. Затем Author выполняет операцию rebase для ветки pull request относительно ветки master. Это может быть выполнено через интерфейс GitHub или с помощью команд: `git checkout <branch_name> && git rebase master && git push -f`. В случае конфликтов Author решает их самостоятельно или в сотрудничестве с Committer и Approver.

### 3.6 Хранение, воспроизведение и выпуск

Данные жизненного цикла (ЖЦ) проекта сохраняются на платформе GitHub, а также в локальных репозиториях каждого из участников проекта. В этом контексте:

- Права доступа к данным ЖЦ устанавливаются и регулируются средствами GitHub. Детальное описание возможных прав доступа представлено в разделе “Организация и ответственность”.
- Данные ЖЦ, которые имеют категорию контроля КК1, сохраняются в git-репозитории.
- Файлы в git-репозитории обеспечивают сохранность и не подвергаются искажению со временем.
- Каждый участник проекта обладает своей локальной копией git-репозитория (возможно, не самой актуальной версии), что обеспечивает дополнительную защиту от потери критически важных данных.
- Данные с категорией контроля КК2 могут быть утрачены в случае потери доступа к облачной платформе GitHub.

**Примечание:** В репозитории содержатся файлы конфигурации и документация к проекту. Документация представлена в виде единого файла формата .pdf. Однако этот файл не хранится напрямую в репозитории. Вместо этого в репозитории находятся исходные файлы и инструкции

для их компиляции в единый документ с использованием LaTeX.

### 3.7 Контроль загрузки ПО

Загрузка ПО не проводится.

### 3.8 Контроль среды ЖЦ

Для обеспечения эффективного управления данными жизненного цикла программного обеспечения используются следующие инструментальные средства:

1. **Draw.io**: Используется для создания и визуализации различных диаграмм, включая архитектурные решения, диаграммы процессов и компонентного взаимодействия.
2. **Golang (Go)**: Язык программирования, выбранный для разработки парсера из-за его высокой производительности и способности обрабатывать большие объемы данных.
3. **PostgreSQL**: Высокопроизводительная система управления базами данных, обеспечивающая надежное хранение, защиту и масштабируемость данных.
4. **IDE**:
  - **GoLand**: Интегрированная среда разработки, специализированная для языка программирования Go. Обеспечивает удобные инструменты для написания, отладки и тестирования кода.
  - **Datagrip**: Инструмент для работы с базами данных, включая PostgreSQL, предоставляющий функции для запросов, анализа и управления данными.
5. **GitHub**: Платформа для хранения исходного кода, обеспечивающая версионный контроль и инструменты для совместной разработки. Все изменения и итерации кода проекта отслеживаются и хранятся на этой платформе.

### 3.9 Средства управления данными ЖЦ ПО

| Единицы конфигурации    | КК1 | КК2 |
|-------------------------|-----|-----|
| Задача                  |     | +   |
| Pull Request            |     | +   |
| Требование              | +   |     |
| Структура               | +   |     |
| Функция                 | +   |     |
| Артефакт проектирования | +   |     |

Трассировочная БД и ПО, обеспечивающее её работу находятся в репозитории, поэтому они хранятся по КК1.

### **3.10 Учёт состояния конфигурации**

Учёт состояния конфигурации обеспечивается посредством использования GitHub и Git.

## **4. Хранение, воспроизведение и выпуск**

В рамках проекта для управления конфигурацией используются следующие документы:

- Сообщения о проблемах (issues) на платформе GitHub.
- Запросы на изменения (pull requests) на платформе GitHub.
- История изменений, документированная в git-репозитории проекта.

## **5. Критерии перехода**

Процесс УК ПО начинается одновременно с процессом планирования создания ПО и заканчивается с выпуском последней версии ПО в рамках курса ТПКС.